

Node.js e npm — Cheat sheet

Note di Emanuele:

Node.js lo stavamo già utilizzando con React e Vite, ma qui vediamo come usarlo da solo, senza Vite.

L'importante ora è concentrarsi su come far partire un progetto Node.js da terminale, come creare script nel file package.json, come installare pacchetti con npm e come importare i moduli JavaScript con `import`.

Pensa a Node.js come JavaScript che gira fuori dal browser, con accesso a file/sistema operativo ed **eseguibile da terminale**. (JavaScript nel browser non ha accesso a file/sistema operativo ma solo al DOM come `document.querySelector()`).

Con Node.js possiamo creare app di ogni tipo con JavaScript, non solo siti web frontend. Useremo Node.js nelle prossime lezioni per creare server e API web che possono essere consumati da app React o siti web con fetch/axios.

Durante lo sviluppo, Node.js gira nella nostra macchina locale, mentre in produzione gira su un server remoto e risponde alle richieste dei client (come app React, siti web).

Concetti essenziali

- **Node.js** è un programma che permette di eseguire JavaScript fuori dal browser, per esempio dal terminale. Per questo viene definito un **ambiente di esecuzione** (o *runtime*) per JavaScript.
- Node.js utilizza il motore V8 di Google Chrome per eseguire il codice JavaScript.
- Nel browser usiamo JavaScript soprattutto per lavorare con una pagina HTML. Con Node.js possiamo invece leggere file, eseguire programmi e, più avanti, creare server e API.
- Node.js può continuare a svolgere altre operazioni mentre aspetta il completamento di un'attività, per esempio la lettura di un file. Il meccanismo che organizza queste attività si chiama **event loop**; per ora è sufficiente conoscerne lo scopo.
- **npm** è lo strumento incluso con Node.js che usiamo per preparare un progetto, installare librerie create da altri sviluppatori ed eseguire i comandi del progetto.

Verificare l'installazione:

```
node --version
npm --version
```

Eseguire JavaScript con Node.js

Avviare un file:

```
node index.js
```

Eeguire una singola istruzione:

```
node -e 'console.log("Hello World")'
```

Aprire la console interattiva **REPL** (*Read, Eval, Print, Loop*):

```
node
```

Per uscire dalla REPL usare `.exit` oppure premere `Ctrl+C` due volte.

Creare un progetto npm

```
mkdir nome-progetto  
cd nome-progetto  
npm init
```

`npm init` crea `package.json` facendo alcune domande. Per accettare subito i valori predefiniti si può usare:

```
npm init -y
```

File e cartelle principali:

- `package.json`: descrive il progetto, gli script e le dipendenze dirette.
- `package-lock.json`: registra le versioni esatte di tutto l'albero delle dipendenze; va normalmente incluso in Git.
- `node_modules/`: contiene i pacchetti installati; va normalmente esclusa da Git con `.gitignore`.

Script npm

Gli script si configurano in `package.json`:

```
{
  "scripts": {
    "start": "node index.js",
    "watch": "node --watch index.js"
  }
}
```

Eseguirli dal terminale:

```
npm start
npm run watch
```

`node --watch` riavvia automaticamente l'applicazione quando un file cambia. Si interrompe con `Ctrl+C`.

`start` è uno script speciale e può essere eseguito senza `run`; per gli altri script si usa `npm run nome-script`.

Moduli JavaScript

Con `"type": "module"` in `package.json` si possono usare gli **ES Modules**:

```
// utils.js
export function saluta(nome) {
  return `Ciao ${nome}`;
}
```

```
// index.js
import { saluta } from './utils.js';

console.log(saluta('Boolean'));
```

Negli import di file locali bisogna indicare il percorso (`./`) e, in Node.js ESM, l'estensione `.js`.

CommonJS è il vecchio sistema di moduli di Node.js e usa `require()` e `module.exports`. In un progetto semplice è meglio non mescolarlo con ESM.

Installare pacchetti

```
npm install nome-pacchetto
npm install --save-dev nome-pacchetto
npm uninstall nome-pacchetto
npm install
```

- Le dipendenze necessarie all'app finiscono in **dependencies**.
- Gli strumenti usati solo durante lo sviluppo finiscono in **devDependencies**.
- **npm install** senza il nome di un pacchetto installa tutte le dipendenze dichiarate dal progetto.

Una versione come **^1.19.0** permette aggiornamenti compatibili della versione **minor** e **patch**, ma non installa automaticamente la major **2.0.0**.

Checklist dell'esercizio

1. Eseguire **npm init** nella cartella del progetto.
2. Creare **index.js** con **console.log('Hello World');**.
3. Aggiungere gli script **start** e **watch** a **package.json**.
4. Avviare il progetto con **npm start**.
5. Avviarlo con **npm run watch**.
6. Cambiare il messaggio in **Hello Boolean** e verificare il riavvio automatico.

Errori comuni

- Lanciare i comandi dalla cartella sbagliata, dove non è presente **package.json**.
- Scrivere JSON non valido in **package.json**: chiavi e stringhe richiedono doppi apici e non sono ammesse virgole finali.
- Dimenticare di salvare il file mentre si prova la modalità watch.
- Confondere **npm start** con **npm run watch**: il primo avvia una volta, il secondo osserva le modifiche.
- Versionare **node_modules/** invece di **package-lock.json**.